

MediByte — Automation QA Assignment (4–6 years experience)

- For assessment use only — MediByte is a sample online pharmacy, not a real service. No real orders are placed and no real data is involved.

About the app

MediByte is an online pharmacy storefront. Customers can browse a catalog of over-the-counter (OTC) and prescription (Rx) products, search and filter the catalog, add items to a cart, apply promo codes, check out with a shipping address, and review their past orders and account details.

Over-the-counter (OTC) vs prescription (Rx)

Pharmacies sell two kinds of product, and the distinction may come up as you explore the app:

- **Over-the-counter (OTC)** — everyday medicines and health products anyone can buy without a doctor's authorization: pain relievers, vitamins, antacids, allergy tablets, first-aid supplies, and the like. In a store like this they behave like ordinary retail items — you pick them and buy them.
- **Prescription (Rx)** — medicines that legally require a valid prescription from a licensed prescriber before they can be dispensed. A pharmacy is expected to treat them differently from OTC items: it should establish that a prescription exists, and it handles the associated patient/health information as sensitive data.

Every product in the catalog is labelled as either OTC or Rx.

Your task

Automate the following user journey through the storefront:

1. Log in and build a cart with at least two different products on it.
2. On the cart page, increase the quantity of one specific line and confirm only that line's quantity and line total change — every other line stays the same.
3. Remove a different line from the cart (confirming the removal when prompted) and confirm only that line disappears, with the subtotal recalculating to match what's left.
4. Apply a promo code and confirm a discount line appears and the total drops by the expected amount, waiting for the page to finish recalculating before you assert.
5. Replace that promo code with a different one and confirm the discount line updates to reflect the new code.
6. Remove the promo code entirely and confirm the total returns to its pre-discount value.
7. Go to checkout, fill in the shipping (and prescription, if required) fields, and submit the order — confirming the submit button shows a pending state while the order is being placed, and separately confirming that leaving a required shipping field blank prevents submission and shows an error instead.

Note that the cart on this page uses repeated controls (one quantity control and one remove control per line), all sharing the same naming pattern across lines — your locators need to be scoped to the specific line you're acting on, not just matched by a generic label.

You do not need to touch any other part of the app — this journey is your complete scope.

Requirements

- **BDD is required.** Write your scenarios in Given/When/Then form (e.g. Cucumber, SpecFlow, Behave, or an equivalent for your language/tool of choice) before or alongside the automation code.
- **Tool choice is free**, but Selenium, Playwright, or Cypress are preferred.
- **Test reporting is required.** Your suite must produce a report artifact (HTML report, Allure report, or similar) that a reviewer can open after a run to see which scenarios passed or failed.
- **The suite must run on any machine.** Provide explicit setup and run instructions (e.g. a [README](#)) that let a reviewer clone your repository, install dependencies, and run the suite without editing any file paths, URLs, or credentials in your code. Any environment-specific value (base URL, login credentials) must be supplied via a config file, environment variable, or command-line argument documented in your instructions — never hardcoded to one machine's file system.

The DevTools Network tab is fair game

The app makes real API calls. You're welcome to open your browser's developer tools and inspect the **Network** tab (and the **Application** / storage tabs, and the **Console**) while you build your automation, if that helps you understand the app's behavior — in particular around when the cart total or coupon state finishes updating after an action.

Test data

Your reviewer will send you a **personal access link** (it looks like <https://.../start?code=...>). Open it once — in the browser your suite drives — and **before you sign in**: it unlocks the app for a limited window, and it is what ties your session to this assignment. If your suite runs in a fresh browser profile each time, make opening that link the first step of your setup, ahead of the login step.

The link is issued for this automation assignment, so the account below is the only one it will sign in with — any other account is rejected.

Sign in with this account. It comes with saved account data and order history so the app has content to work with.

Email	Password
steve@example.test	steve1234

There is no self-registration for this assessment — always use this account.

These promo codes can be applied in the cart while you build your automation:

Code	What it offers	Minimum order	Notes
SAVE10	10% off your order	none	
WELCOMES	\$5 off your order	\$25	only applies at \$25+
WELLNESS15	15% off your order	\$40	only applies at \$40+
MEGA50	\$50 off your order	none	
SPRING2023	20% off (spring promo)	none	expired on 2023-05-31

Submit your automation project (source code, BDD feature files, and a generated test report from a run) along with a short note on any assumptions you made or issues you noticed along the way. The clarity and structure of your project is part of what we assess, alongside whether the suite runs successfully on our end.